

هوش مصنوعی و کارگاه

دانشکده ریاضی و علوم کامپیوتر دانشگاه صنعتی امیرکبیر



مدرس کلاس: مهدی قطعی

استاد گروه علوم کامپیوتر

www.aut.ac.ir

مدرس کارگاه: بهنام یوسفی مهر

دانشجوی دکتری علوم کامپیوتر

Artificial Intelligence & Workshop

روشهای طبقه بندی (درخت تصمیم، شبکه
بیزی، ماشین بردار پشتیبان، یادگیری ترکیبی،
جنگل تصادفی)

Ref:

Data Mining: Concepts and Techniques

(3rd ed.)

— Chapter 8 —

Jiawei Han, Micheline Kamber, and Jian Pei
University of Illinois at Urbana-Champaign &
Simon Fraser University

©2011 Han, Kamber & Pei. All rights reserved.

Classification: Basic Concepts

- Classification: Basic Concepts 
- Decision Tree Induction
- Bayesian Network
- Support Vector Machine (SVM)
- CF-Tree: Hierarchical Micro-cluster
- Ensemble Learning
- Random Forest

Supervised vs. Unsupervised Learning

- **Supervised learning (classification)**
 - Supervision: The training data (observations, measurements, etc.) are accompanied by **labels** indicating the class of the observations
 - New data is classified based on the training set
- **Unsupervised learning (clustering)**
 - The class labels of training data is unknown
 - Given a set of measurements, observations, etc. with the aim of establishing the existence of classes or clusters in the data

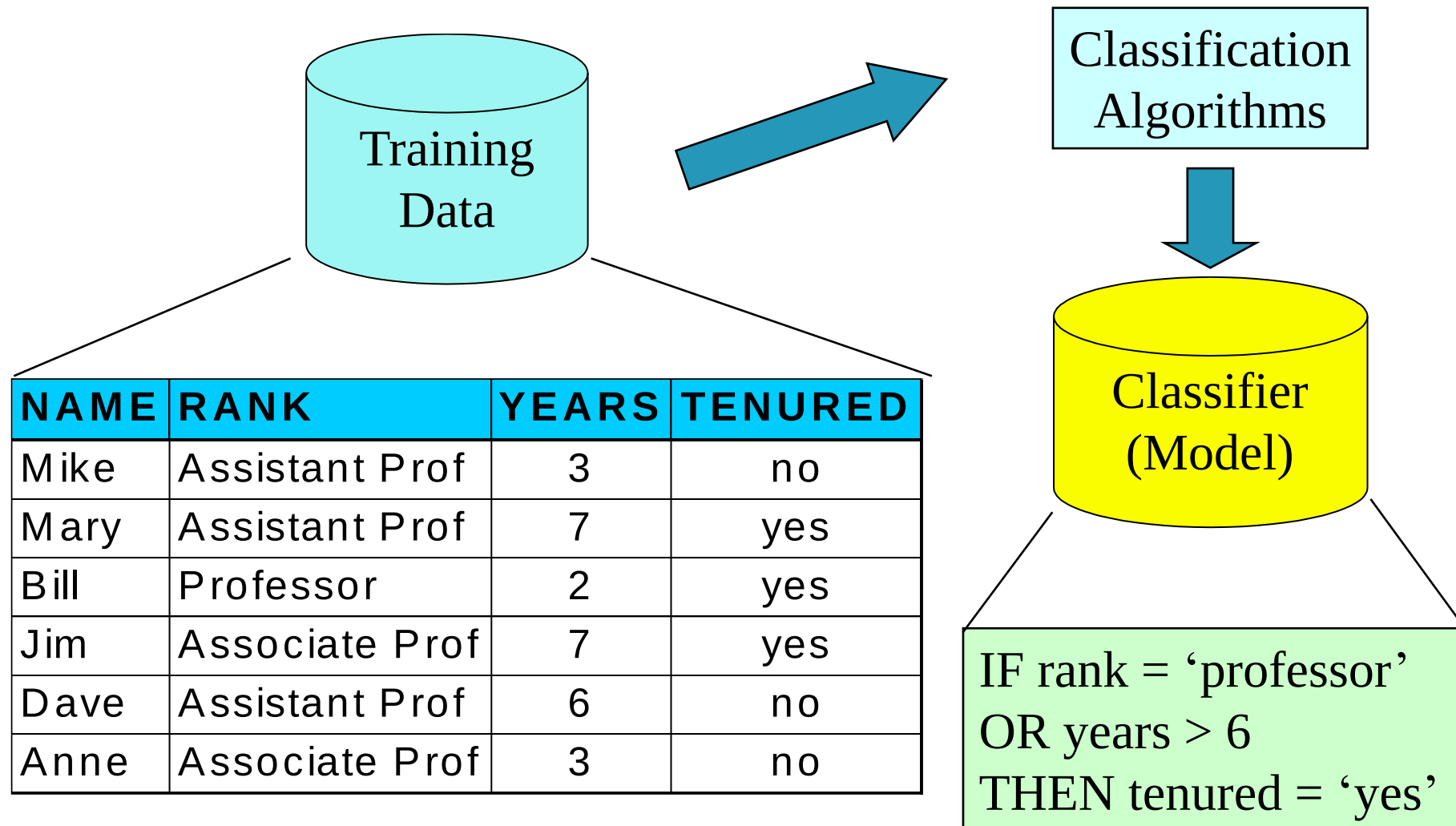
Prediction Problems: Classification vs. Numeric Prediction

- **Classification**
 - predicts categorical class labels (discrete or nominal)
 - classifies data (constructs a model) based on the training set and the values (**class labels**) in a classifying attribute and uses it in classifying new data
- **Numeric Prediction**
 - models continuous-valued functions, i.e., predicts unknown or missing values
- **Typical applications**
 - Credit/loan approval:
 - Medical diagnosis: if a tumor is cancerous or benign
 - Fraud detection: if a transaction is fraudulent
 - Web page categorization: which category it is

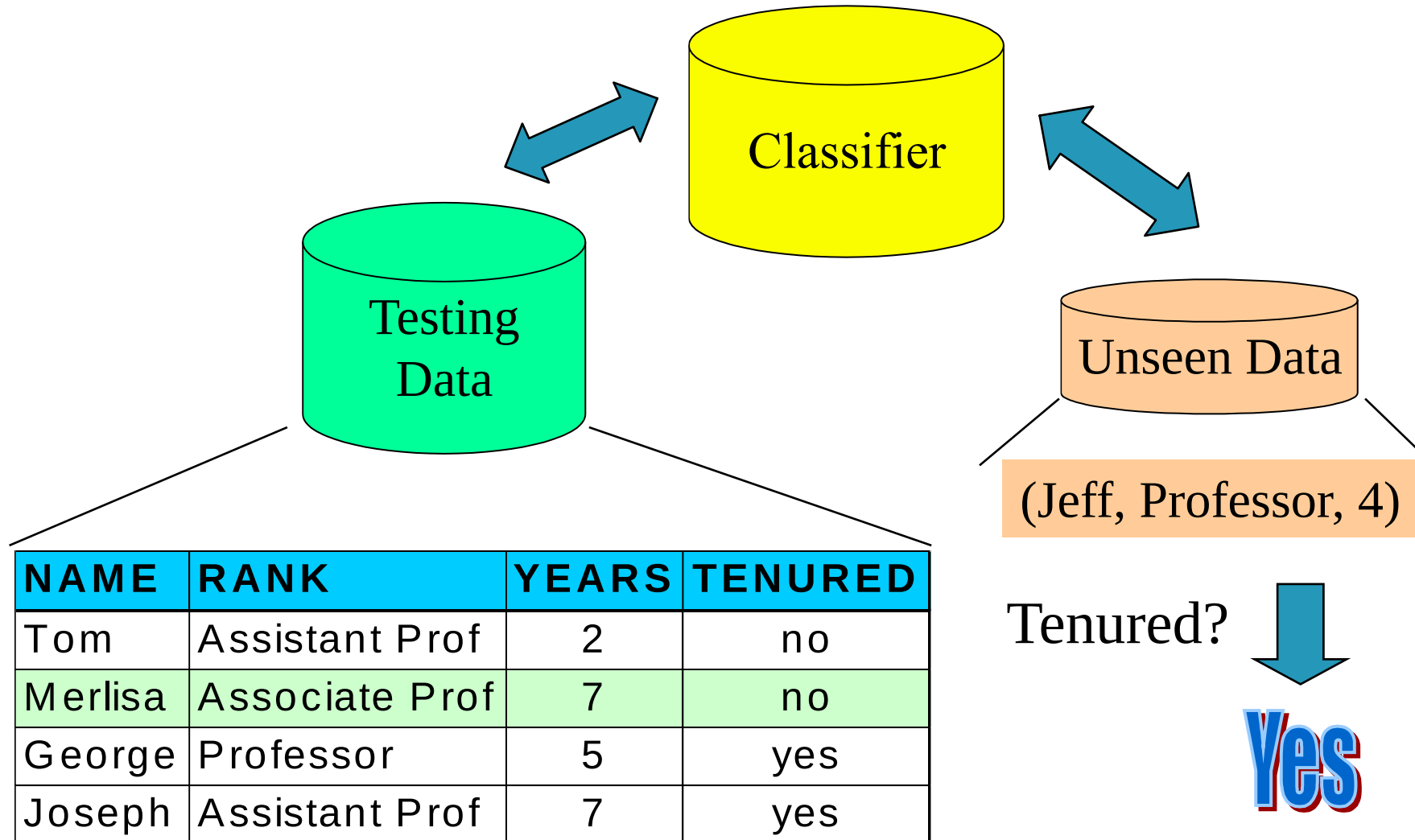
Classification—A Two-Step Process

- **Model construction:** describing a set of predetermined classes
 - Each tuple/sample is assumed to belong to a predefined class, as determined by the **class label attribute**
 - The set of tuples used for model construction is **training set**
 - The model is represented as classification rules, decision trees, or mathematical formulae
- **Model usage:** for classifying future or unknown objects
 - **Estimate accuracy** of the model
 - The known label of test sample is compared with the classified result from the model
 - **Accuracy** rate is the percentage of test set samples that are correctly classified by the model
 - **Test set** is independent of training set (otherwise overfitting)
 - If the accuracy is acceptable, use the model to **classify new data**
- **Note:** If *the test set* is used to select models, it is called **validation (test) set**

Process (1): Model Construction



Process (2): Using the Model in Prediction



Classification: Basic Concepts

- Classification: Basic Concepts
- Decision Tree Induction 
- Bayesian Network
- Support Vector Machine (SVM)
- CF-Tree: Hierarchical Micro-cluster
- Ensemble Learning
- Random Forest

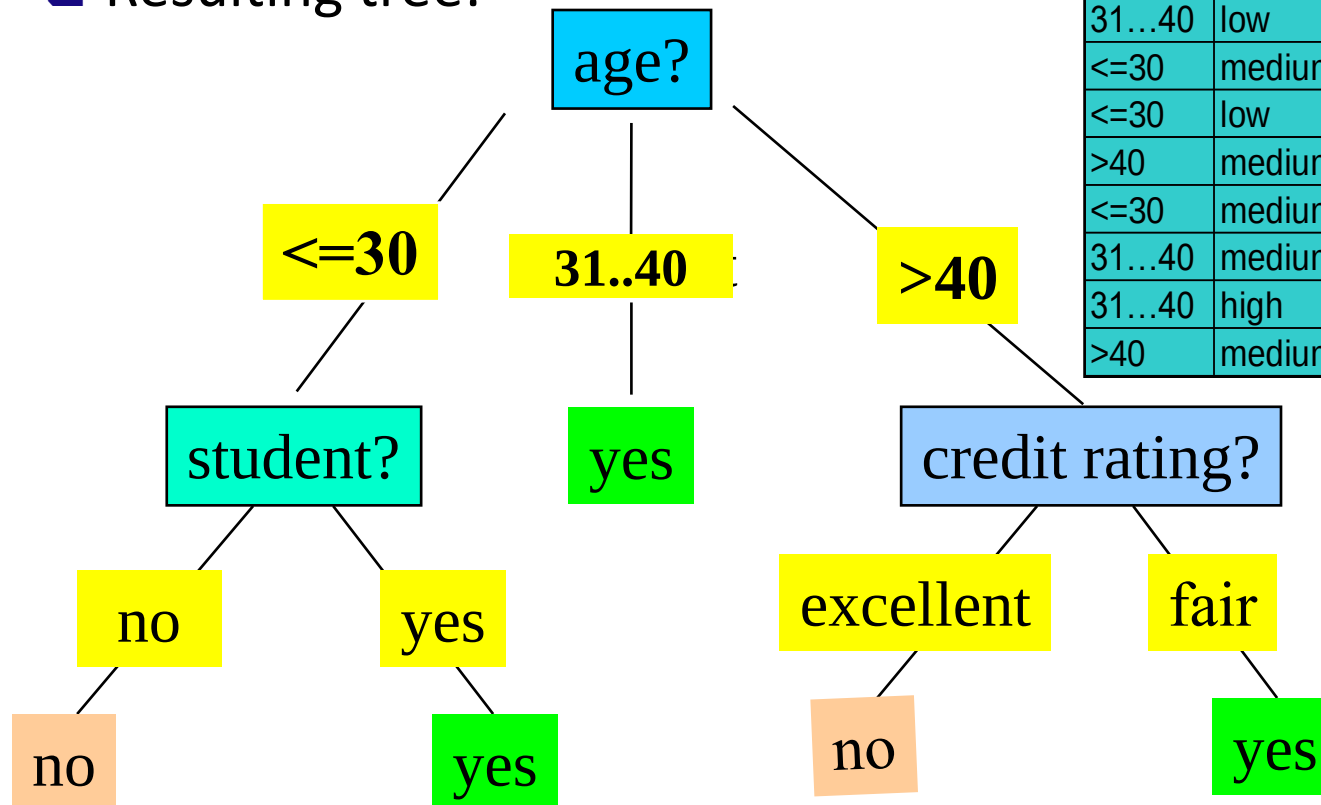
John Ross Quinlan is a [computer science](#) researcher in [data mining](#) and [decision theory](#). He has contributed extensively to the development of [decision tree](#) algorithms, including inventing the canonical [C4.5](#) and [ID3](#) algorithms.



TITLE	CITED BY	YEAR
C4. 5: Programs for machine learning JR Quinlan Morgan Kaufmann, San Francisco, CA	46122 *	1993
Induction of decision trees JR Quinlan Machine learning 1 (1), 81-106	34340	1986
Top 10 algorithms in data mining X Wu, V Kumar, J Ross Quinlan, J Ghosh, Q Yang, H Motoda, ... Knowledge and information systems 14 (1), 1-37	8063	2008
Learning with continuous classes JR Quinlan 5th Australian joint conference on artificial intelligence 92, 343-348	4433	1992

Decision Tree Induction: An Example

- ❑ Training data set: Buys_computer
- ❑ The data set follows an example of Quinlan's ID3
- ❑ Resulting tree:



age	income	student	credit_rating	buys_computer
<=30	high	no	fair	no
<=30	high	no	excellent	no
31...40	high	no	fair	yes
>40	medium	no	fair	yes
>40	low	yes	fair	yes
>40	low	yes	excellent	no
31...40	low	yes	excellent	yes
<=30	medium	no	fair	no
<=30	low	yes	fair	yes
>40	medium	yes	fair	yes
<=30	medium	yes	excellent	yes
31...40	medium	no	excellent	yes
31...40	high	yes	fair	yes
>40	medium	no	excellent	no

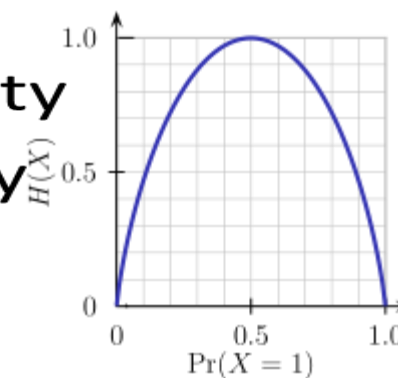
Algorithm for Decision Tree Induction

- Basic algorithm (a greedy algorithm)
 - Tree is constructed in a **top-down recursive divide-and-conquer manner**
 - At start, all the training examples are at the root
 - Attributes are categorical (if continuous-valued, they are **discretized** in advance)
 - Examples are partitioned recursively based on selected attributes
 - Test attributes are selected on the basis of a heuristic or statistical measure (e.g., **information gain**)
- Conditions for stopping partitioning
 - All samples for a given node belong to the same class
 - There are no remaining attributes for further partitioning – **majority voting** is employed for classifying the leaf
 - There are no samples left

Brief Review of Entropy

■ Entropy (Information Theory)

- A measure of uncertainty associated with a random variable
- Calculation: For a discrete random variable Y taking m distinct values $\{y_1, \dots, y_m\}$,
 - $H(Y) = -\sum_{i=1}^m p_i \log(p_i)$, where $p_i = P(Y = y_i)$
- Interpretation:
 - Higher entropy $\Rightarrow$ higher uncertainty
 - Lower entropy $\Rightarrow$ lower uncertainty



m = 2

■ Conditional Entropy

- $H(Y|X) = \sum_x p(x)H(Y|X = x)$

Example of Entropy

- Example: Consider two experiments:
 - Experiment A (less random): A fake coin that comes up heads 90% of the time and tails 10% of the time:
 - $H(A) = - [0.9 * \log_2(0.9) + 0.1 * \log_2(0.1)] \approx 0.47$
 - Experiment B (highly random): A perfectly fair coin that comes up heads 50% of the time and tails 50% of the time:
 - $H(B) = - [0.5 * \log_2(0.5) + 0.5 * \log_2(0.5)] = 1$
 - Result: The entropy of experiment B is clearly greater than that of experiment.
 - In experiment A with a fake coin, you are almost certain that the outcome will be heads. So there is little uncertainty and surprise.

Attribute Selection Measure: Information Gain (ID3/C4.5)

- Select the attribute with the highest information gain
- Let p_i be the probability that an arbitrary tuple in D belongs to class C_i , estimated by $|C_{i,D}|/|D|$

- Expected information (entropy) needed to classify a tuple in D :

$$Info(D) = -\sum_{i=1}^m p_i \log_2(p_i)$$

- Information needed (after using A to split D into v partitions) to classify D :

$$Info_A(D) = \sum_{j=1}^v \frac{|D_j|}{|D|} \times Info(D_j)$$

- Information gained by branching on attribute A

$$Gain(A) = Info(D) - Info_A(D)$$

Why is a Larger Gain Better?

- Information Gain is calculated precisely based on the **reduction in entropy**: $Gain(A) = Info(D) - Info_A(D)$

where:

- $Info(D)$: Entropy before the split
- $Gain(A)$: Entropy after splitting based on feature A
- **Greater Reduction in Disorder**: When a feature's Gain is large, it means:
 - **Entropy has decreased significantly after the split**
 - The data is better classified
 - There is less impurity in the subsets

Attribute Selection: Information Gain

- Class P: buys_computer = "yes"
- Class N: buys_computer = "no"

$$Info(D) = I(9,5) = -\frac{9}{14} \log_2\left(\frac{9}{14}\right) - \frac{5}{14} \log_2\left(\frac{5}{14}\right) = 0.940$$

age	p _i	n _i	I(p _i , n _i)
<=30	2	3	0.971
31...40	4	0	0
>40	3	2	0.971

age	income	student	credit_rating	buys_computer
<=30	high	no	fair	no
<=30	high	no	excellent	no
31...40	high	no	fair	yes
>40	medium	no	fair	yes
>40	low	yes	fair	yes
>40	low	yes	excellent	no
31...40	low	yes	excellent	yes
<=30	medium	no	fair	no
<=30	low	yes	fair	yes
>40	medium	yes	fair	yes
<=30	medium	yes	excellent	yes
31...40	medium	no	excellent	yes
31...40	high	yes	fair	yes
>40	medium	no	excellent	no

$$Info_{age}(D) = \frac{5}{14} I(2,3) + \frac{4}{14} I(4,0) + \frac{5}{14} I(3,2) = 0.694$$

$\frac{5}{14} I(2,3)$ means "age <=30" has 5 out of 14 samples, with 2 yes'es and 3 no's. Hence

$$Gain(age) = Info(D) - Info_{age}(D) = 0.246$$

Similarly,

$$Gain(income) = 0.029$$

$$Gain(student) = 0.151$$

$$Gain(credit_rating) = 0.048$$

Computing Information-Gain for Continuous-Valued Attributes

- Let attribute A be a continuous-valued attribute
- Must determine the *best split point* for A
 - Sort the value A in increasing order
 - Typically, the midpoint between each pair of adjacent values is considered as a possible *split point*
 - $(a_i + a_{i+1})/2$ is the midpoint between the values of a_i and a_{i+1}
 - For each possible split-point for A , we evaluate $\text{Info}_A(D)$, (# of partitions=2). The point with the minimum expected information requirement, $\text{Info}_A(D)$, for A is selected as the split point for A .
- Split:
 - D_1 is the set of tuples in D satisfying $A \leq \text{split-point}$, and D_2 is the set of tuples in D satisfying $A > \text{split-point}$

Gain Ratio for Attribute Selection (C4.5)

- Information gain measure is biased towards attributes with a large number of values
- C4.5 (a successor of ID3) uses gain ratio to overcome the problem (normalization to information gain)

$$SplitInfo_A(D) = - \sum_{j=1}^v \frac{|D_j|}{|D|} \times \log_2 \left(\frac{|D_j|}{|D|} \right)$$

- GainRatio(A) = Gain(A)/SplitInfo(A)
- Ex. $SplitInfo_{income}(D) = -\frac{4}{14} \times \log_2 \left(\frac{4}{14} \right) - \frac{6}{14} \times \log_2 \left(\frac{6}{14} \right) - \frac{4}{14} \times \log_2 \left(\frac{4}{14} \right) = 1.557$
 - gain_ratio(income) = 0.029/1.557 = 0.019
- The attribute with the maximum gain ratio is selected as the splitting attribute

Gini Index (CART, IBM IntelligentMiner)

- If a data set D contains examples from n classes, gini index, $gini(D)$ is defined as

$$gini(D) = 1 - \sum_{j=1}^n p_j^2$$

where p_j is the relative frequency of class j in D

- If a data set D is split on A into two subsets D_1 and D_2 , the gini index $gini(D)$ is defined as

$$gini_A(D) = \frac{|D_1|}{|D|} gini(D_1) + \frac{|D_2|}{|D|} gini(D_2)$$

- Reduction in Impurity:

$$\Delta gini(A) = gini(D) - gini_A(D)$$

- The attribute provides the smallest $gini_{split}(D)$ (or the largest reduction in impurity) is chosen to split the node (*need to enumerate all the possible splitting points for each attribute*)

Computation of Gini Index

income	p	n
Low	3	1
Medium	4	2
high	2	2

- Ex. D has 9 tuples in buys_computer = “yes” and 5 in “no”

$$gini(D) = 1 - \left(\frac{9}{14}\right)^2 - \left(\frac{5}{14}\right)^2 = 0.459$$

- Suppose the attribute income partitions D into 10 in D_1 : {low, medium} and 4 in D_2

$$\begin{aligned}
 gini_{income \in \{low, medium\}}(D) &= \left(\frac{10}{14}\right)Gini(D_1) + \left(\frac{4}{14}\right)Gini(D_2) \\
 &= \frac{10}{14} \left(1 - \left(\frac{7}{10}\right)^2 - \left(\frac{3}{10}\right)^2\right) + \frac{4}{14} \left(1 - \left(\frac{2}{4}\right)^2 - \left(\frac{2}{4}\right)^2\right) \\
 &= 0.443 \\
 &= Gini_{income \in \{high\}}(D).
 \end{aligned}$$

$Gini_{\{low, high\}}$ is 0.458; $Gini_{\{medium, high\}}$ is 0.450. Thus, split on the {low, medium} (and {high}) since it has the lowest Gini index

- All attributes are assumed continuous-valued
- May need other tools, e.g., clustering, to get the possible split values
- Can be modified for categorical attributes

Comparing Attribute Selection Measures

- The three measures, in general, return good results but
 - **Information gain:**
 - biased towards multivalued attributes
 - **Gain ratio:**
 - tends to prefer unbalanced splits in which one partition is much smaller than the others
 - **Gini index:**
 - biased to multivalued attributes
 - has difficulty when # of classes is large
 - tends to favor tests that result in equal-sized partitions and purity in both partitions

Overfitting in Decision Tree

- Errors committed by classification models are generally divided into two types:

Training Errors

The number of misclassification errors committed on training records; also called resubstitution error.

Generalization Errors

The expected error of the model on previously unseen records.

- When **Generalization Errors > Training Errors**, refers overfitting.

Causes of Overfitting

- **Overfitting Due to Presence of Noise**

Mislabeled instances may contradict the class labels of other similar records.

Overfitting Due to Lack of Representative Instances

Lack of representative instances in the training data can prevent refinement of the learning algorithm.

Overfitting and the Multiple Comparison Procedure

Failure to compensate for algorithms that explore a large number of alternatives can result in spurious fitting.

Overfitting in Decision Tree

An example training set for classifying mammals. Asterisks denote mislabelings.

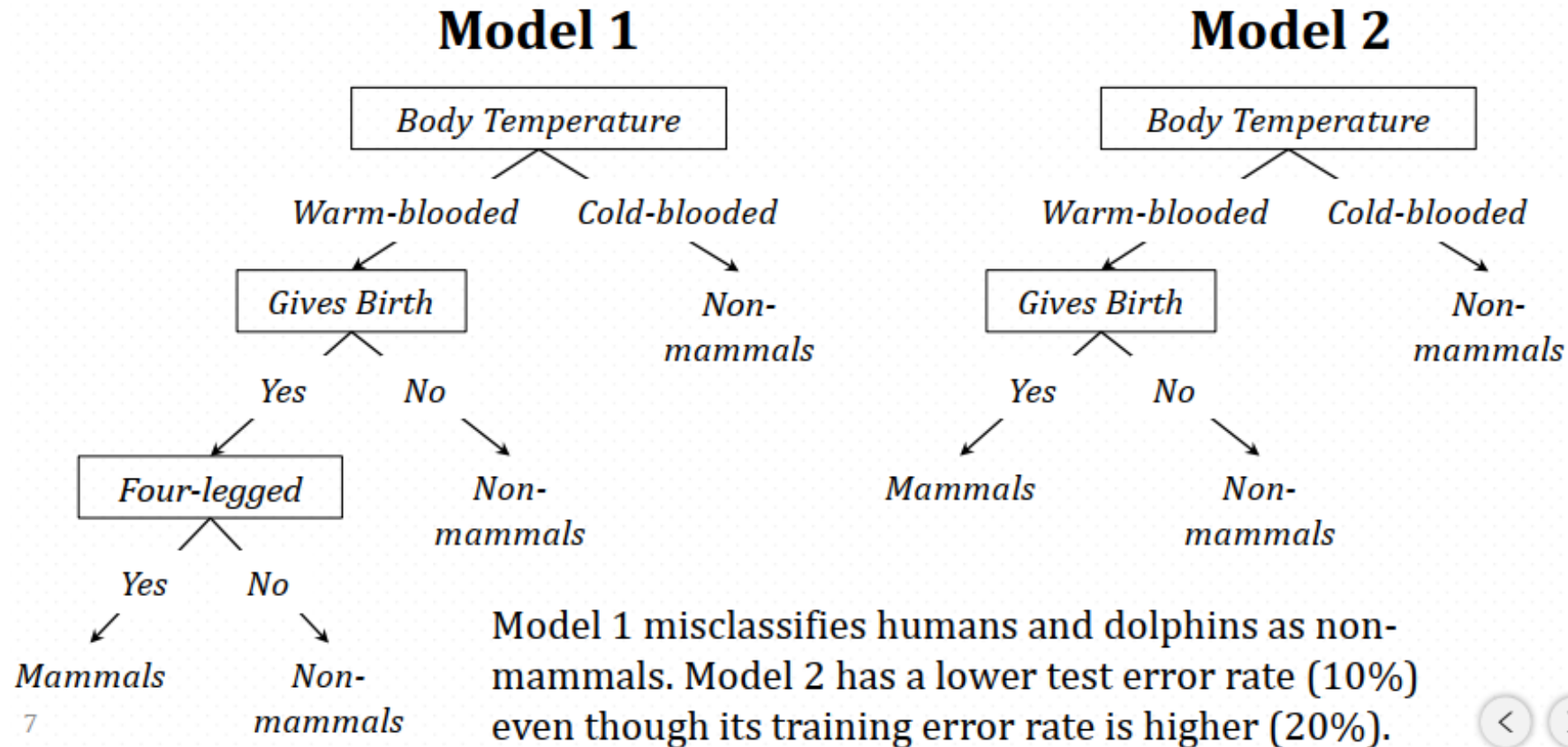
Name	Body Temperature	Gives Birth	Four-legged	Hibernates	Class Label
Porcupine	Warm-blooded	Yes	Yes	Yes	<i>Yes</i>
Cat	Warm-blooded	Yes	Yes	No	<i>Yes</i>
Bat	Warm-blooded	Yes	No	Yes	<i>No*</i>
Whale	Warm-blooded	Yes	No	No	<i>No*</i>
Salamander	Cold-blooded	No	Yes	Yes	<i>No</i>
Komodo dragon	Cold-blooded	No	Yes	No	<i>No</i>
Python	Cold-blooded	No	No	Yes	<i>No</i>
Salmon	Cold-blooded	No	No	No	<i>No</i>
Eagle	Warm-blooded	No	No	No	<i>No</i>
Guppy	Cold-blooded	Yes	No	No	<i>No</i>

Overfitting in Decision Tree

An example testing set for classifying mammals.

Name	Body Temperature	Gives Birth	Four-legged	Hibernates	Class Label
Human	Warm-blooded	Yes	No	No	<i>Yes</i>
Pigeon	Warm-blooded	No	No	No	<i>No</i>
Elephant	Warm-blooded	Yes	Yes	No	<i>Yes</i>
Leopard shark	Cold-blooded	Yes	No	No	<i>No</i>
Turtle	Cold-blooded	No	Yes	No	<i>No</i>
Penguin	Cold-blooded	No	No	No	<i>No</i>
Eel	Cold-blooded	No	No	No	<i>No</i>
Dolphin	Warm-blooded	Yes	No	No	<i>Yes</i>
Spiny anteater	Warm-blooded	No	Yes	Yes	<i>Yes</i>
Gila monster	Cold-blooded	No	Yes	Yes	<i>No</i>

Overfitting in Decision Tree



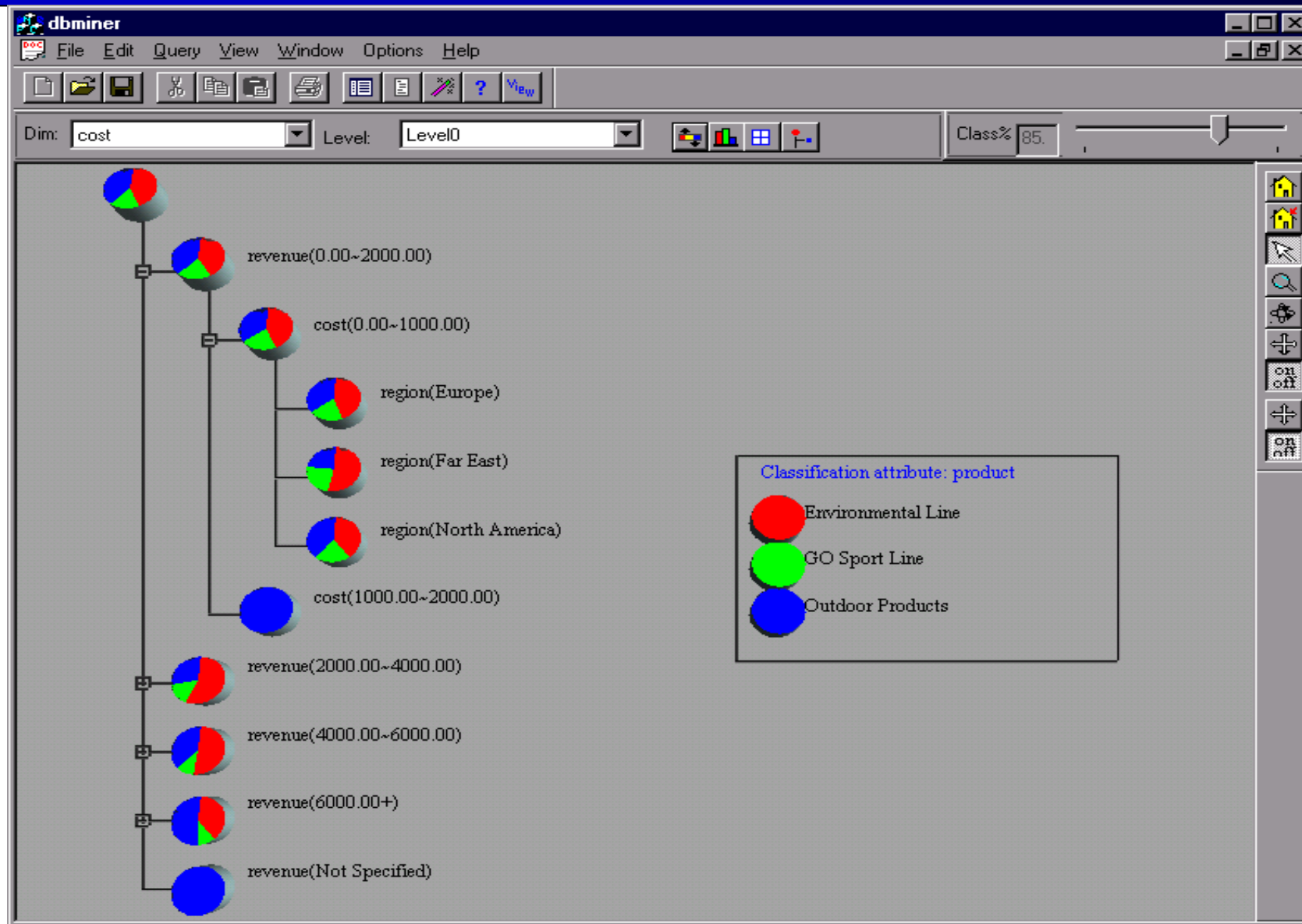
Overfitting and Tree Pruning

- Overfitting: An induced tree may overfit the training data
 - Too many branches, some may reflect anomalies due to noise or outliers
 - Poor accuracy for unseen samples
- Two approaches to avoid overfitting
 - Prepruning: *Halt tree construction early*—do not split a node if this would result in the goodness measure falling below a threshold
 - Difficult to choose an appropriate threshold
 - Postpruning: *Remove branches* from a “fully grown” tree—get a sequence of progressively pruned trees
 - Use a set of data different from the training data to decide which is the “best pruned tree”

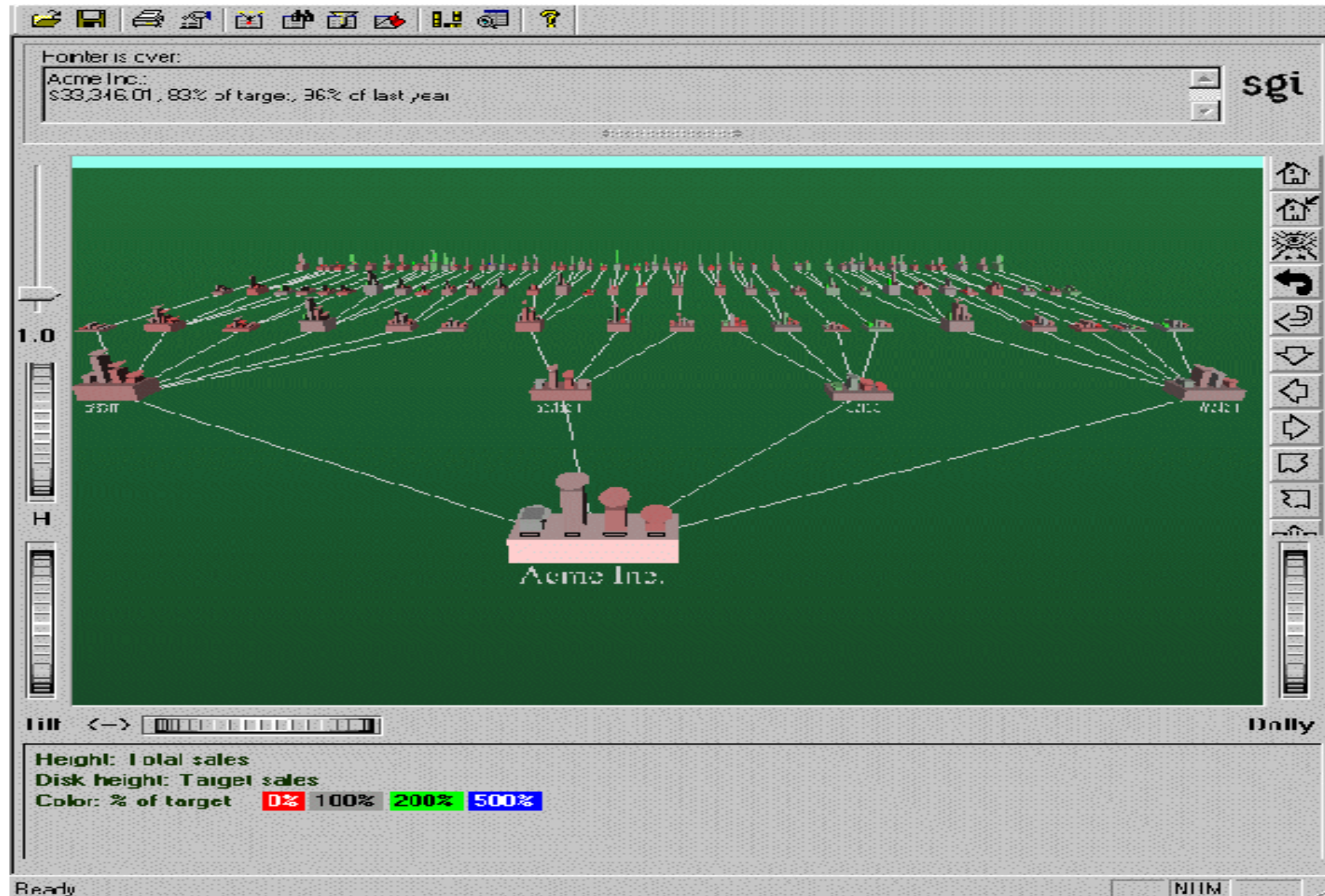
Enhancements to Basic Decision Tree Induction

- Allow for **continuous-valued attributes**
 - Dynamically define new discrete-valued attributes that partition the continuous attribute value into a discrete set of intervals
- Handle **missing attribute values**
 - Assign the most common value of the attribute
 - Assign probability to each of the possible values
- **Attribute construction**
 - Create new attributes based on existing ones that are sparsely represented
 - This reduces fragmentation, repetition, and replication

Presentation of Classification Results



Visualization of a Decision Tree in SGI/MineSet 3.0



Classification: Basic Concepts

- Classification: Basic Concepts
- Decision Tree Induction
- Bayesian Network 
- Support Vector Machine (SVM)
- CF-Tree: Hierarchical Micro-cluster
- Ensemble Learning
- Random Forest

Bayesian networks

- A "Map of Cause and Effect"
- Imagine you want to figure out why your grass is wet in the morning.
 - You know two possible causes:
 - The sprinkler ran last night.
 - It rained last night.
 - You also know that rain can affect whether the sprinkler was turned on (people often don't water the lawn if it's going to rain).

Bayesian networks

- A Bayesian Network is a diagram that maps out these cause-and-effect relationships, and it uses the math of probability to answer questions about them.
- Here's how it works:
 - It's a Network of Nodes: Each circle (a "node") represents a piece of the puzzle. In our example, the nodes are: Rain, Sprinkler, and Wet Grass.
 - Arrows Show Influence: The arrows between the nodes show how they influence each other.

Bayesian networks,...

- Network contains:
 - Rain $\rightarrow$ Sprinkler (Rain influences the chance the sprinkler was on)
 - Rain $\rightarrow$ Wet Grass
 - Sprinkler $\rightarrow$ Wet Grass
- It Handles Uncertainty with Probability: For each node, the network holds a little table of probabilities.
 - The Rain node might say: "There's a 10% chance it rained."
 - The Sprinkler node would say: "If it rained, there's a 1% chance the sprinkler was on. If it didn't rain, there's a 40% chance it was on."

Inference

- The Magic: It Can Reason in Any Direction
- This is the powerful part. Once the network is built, you can ask it questions, and it will use probability to give you the most likely answer.
- Observation: "The grass is wet. What's the most likely cause?"
 - Inference: The network calculates: "Given the grass is wet, there is a 75% chance it rained and a 25% chance just the sprinkler was on."
- Observation: "The grass is wet, and I know the sprinkler was definitely OFF."
 - Inference: The network updates: "Then there is a 99% chance it rained."

Bayesian Network

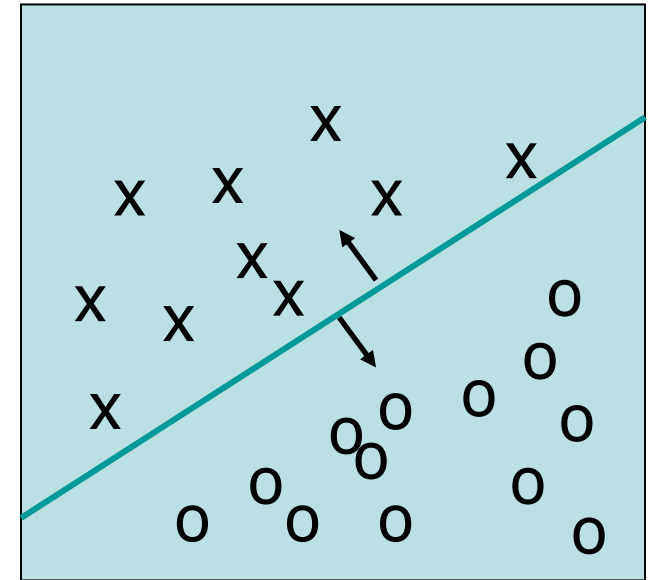
- In short:
- A Bayesian Network is a visual tool that uses probability to model real-world relationships. It lets you reason about causes and effects, even when you have incomplete information.

Classification: Basic Concepts

- Classification: Basic Concepts
- Decision Tree Induction
- Bayesian Network
- Support Vector Machine (SVM) 
- CF-Tree: Hierarchical Micro-cluster
- Ensemble Learning
- Random Forest

Classification: A Mathematical Mapping

- **Classification:** predicts categorical class labels
 - E.g., Personal homepage classification
 - $x_i = (x_1, x_2, x_3, \dots)$, $y_i = +1$ or -1
- Mathematically, $x \in X = \mathbb{R}^n$, $y \in Y = \{+1, -1\}$,
 - We want to derive a function $f: X \rightarrow Y$
- **Linear Classification**
 - Binary Classification problem
 - Data above the green line belongs to class 'x'
 - Data below green line belongs to class 'o'
 - Examples: SVM, Perceptron, Probabilistic Classifiers



Discriminative Classifiers

- Advantages

- Prediction accuracy is generally high
 - As compared to Bayesian methods – in general
- Robust, works when training examples contain errors
- Fast evaluation of the learned target function
 - Bayesian networks are normally slow

- Criticism

- Long training time
- Difficult to understand the learned function (weights)
 - Bayesian networks can be used easily for pattern discovery
- Not easy to incorporate domain knowledge
 - Easy in the form of priors on the data or distributions

SVM—Support Vector Machines

- A relatively new classification method for both linear and nonlinear data
- It uses a nonlinear mapping to transform the original training data into a higher dimension
- With the new dimension, it searches for the linear optimal separating **hyperplane** (i.e., “decision boundary”)
- With an appropriate nonlinear mapping to a sufficiently high dimension, data from two classes can always be separated by a hyperplane
- SVM finds this hyperplane using **support vectors** (“essential” training tuples) and **margins** (defined by the support vectors)

SVM—Support Vector Machines

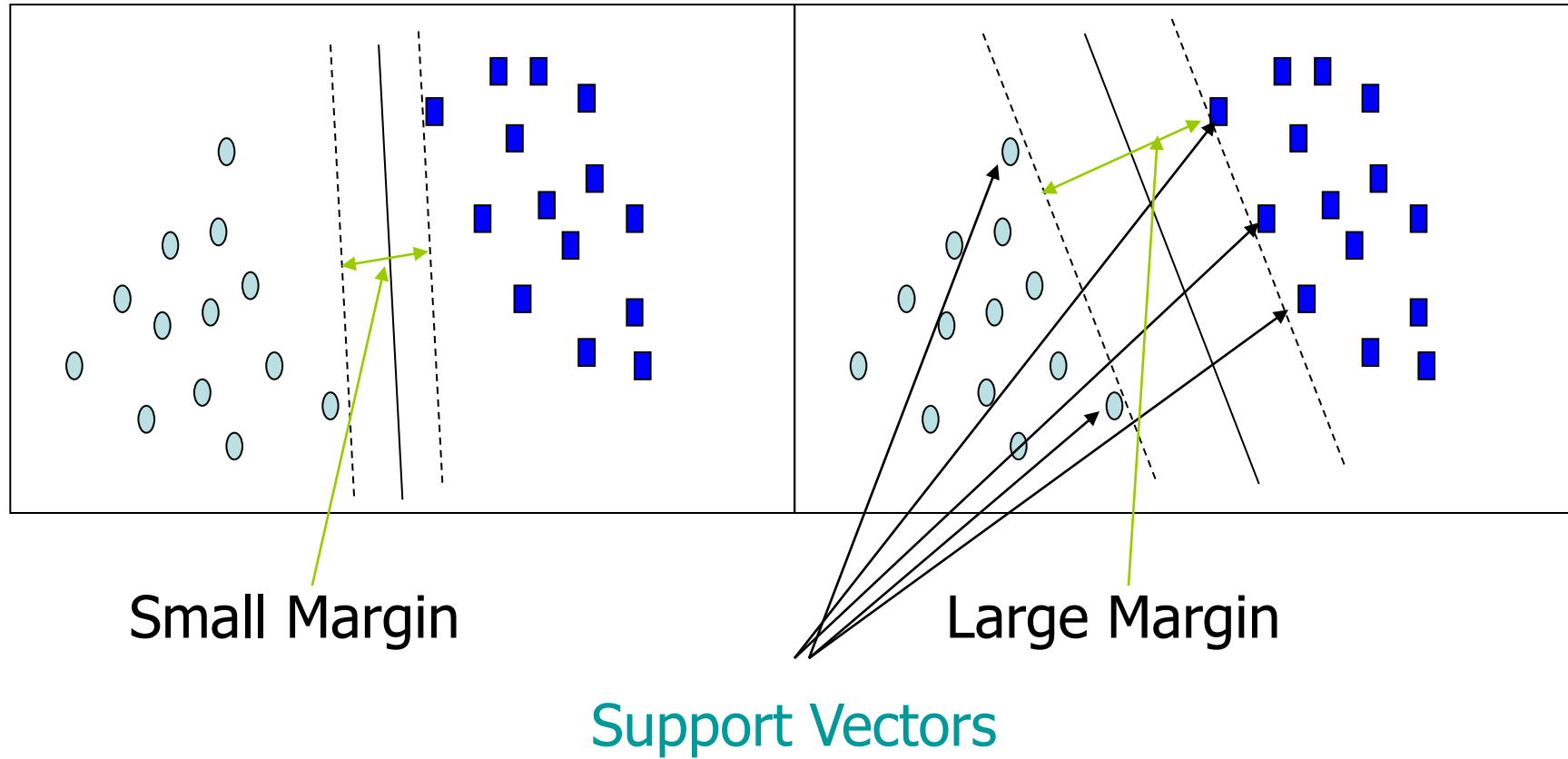
- **The "Street in the Middle" Classifier**
- Imagine you have two types of data points on a page: **Circles** and **Squares**. Your goal is to draw a single, straight line that best separates them.
- A **Support Vector Machine (SVM)** does this with one key goal in mind:
- **Find the widest possible "street" you can draw between the closest Circles and Squares.**

SVM—Support Vector Machines

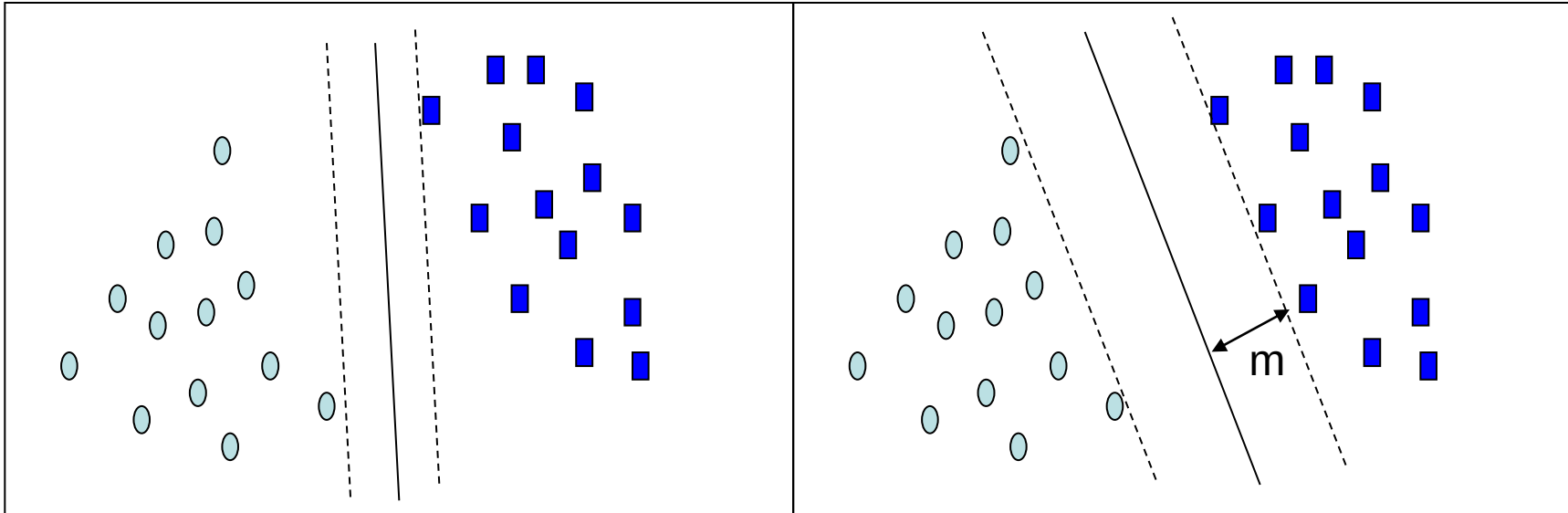
- **The Line:** The perfect center line of this street is the **decision boundary** that separates the two classes.
- **The Borders:** The edges of the street are drawn so that they touch the closest data points from each class. These critical points are called the **Support Vectors** (they "support" the street's borders).
- **The Width:** The space between the borders is the **margin**. The SVM isn't just trying to find any separating line; it's specifically trying to find the line that **maximizes the margin**.

-
- **Why a Wide Street?**
 - A wider street is better because it creates a more robust and general-purpose classifier. A new data point that falls clearly on one side of the street is confidently classified. This focus on the most difficult, borderline points (the Support Vectors) makes SVMs very effective, especially in complex situations.
 - **In short: An SVM finds the best "decision street" by focusing only on the data points that are hardest to separate, and it ignores everything else.**

SVM—General Philosophy



SVM—When Data Is Linearly Separable



Let data D be $(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_{|D|}, y_{|D|})$, where $\mathbf{x}_i$ is the set of training tuples associated with the class labels y_i

There are infinite lines (hyperplanes) separating the two classes but we want to find the best one (the one that minimizes classification error on unseen data)

*SVM searches for the hyperplane with the largest margin, i.e., **maximum marginal hyperplane** (MMH)*

SVM—Linearly Separable

- A separating hyperplane can be written as

$$\mathbf{W} \bullet \mathbf{X} + b = 0$$

where $\mathbf{W} = \{w_1, w_2, \dots, w_n\}$ is a weight vector and b a scalar (bias)

- For 2-D it can be written as

$$w_0 + w_1 x_1 + w_2 x_2 = 0$$

- The hyperplane defining the sides of the margin:

$$H_1: w_0 + w_1 x_1 + w_2 x_2 \geq 1 \quad \text{for } y_i = +1, \text{ and}$$

$$H_2: w_0 + w_1 x_1 + w_2 x_2 \leq -1 \text{ for } y_i = -1$$

- Any training tuples that fall on hyperplanes H_1 or H_2 (i.e., the sides defining the margin) are **support vectors**
- This becomes a **constrained (convex) quadratic optimization** problem: Quadratic objective function and linear constraints $\rightarrow$ *Quadratic Programming (QP)* $\rightarrow$ Lagrangian multipliers

SVM: Different Kernel functions

- Instead of computing the dot product on the transformed data, it is math. equivalent to applying a kernel function $K(\mathbf{X}_i, \mathbf{X}_j)$ to the original data, i.e., $K(\mathbf{X}_i, \mathbf{X}_j) = \Phi(\mathbf{X}_i) \cdot \Phi(\mathbf{X}_j)$
- Typical Kernel Functions

Polynomial kernel of degree h : $K(\mathbf{X}_i, \mathbf{X}_j) = (\mathbf{X}_i \cdot \mathbf{X}_j + 1)^h$

Gaussian radial basis function kernel : $K(\mathbf{X}_i, \mathbf{X}_j) = e^{-\|\mathbf{X}_i - \mathbf{X}_j\|^2 / 2\sigma^2}$

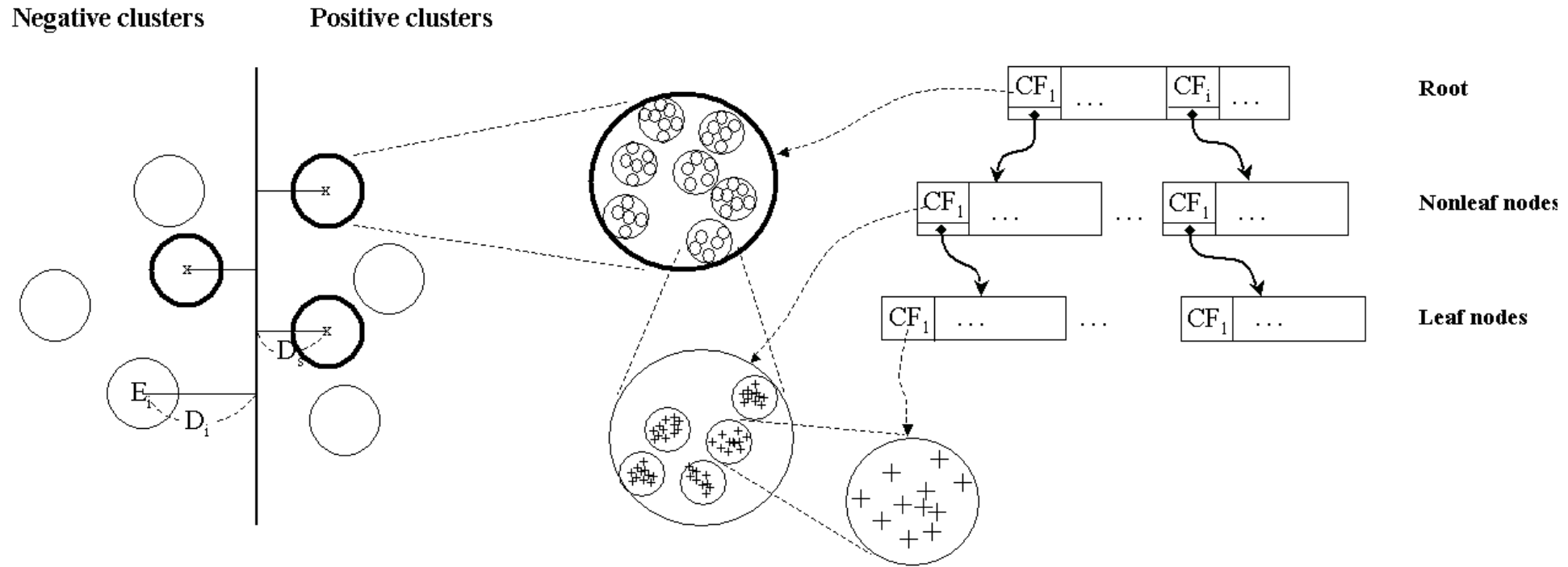
Sigmoid kernel : $K(\mathbf{X}_i, \mathbf{X}_j) = \tanh(\kappa \mathbf{X}_i \cdot \mathbf{X}_j - \delta)$

- SVM can also be used for classifying multiple (> 2) classes and for regression analysis (with additional parameters)

Classification: Basic Concepts

- Classification: Basic Concepts
- Decision Tree Induction
- Bayesian Network
- Support Vector Machine (SVM)
- CF-Tree: Hierarchical Micro-cluster 
- Ensemble Learning
- Random Forest

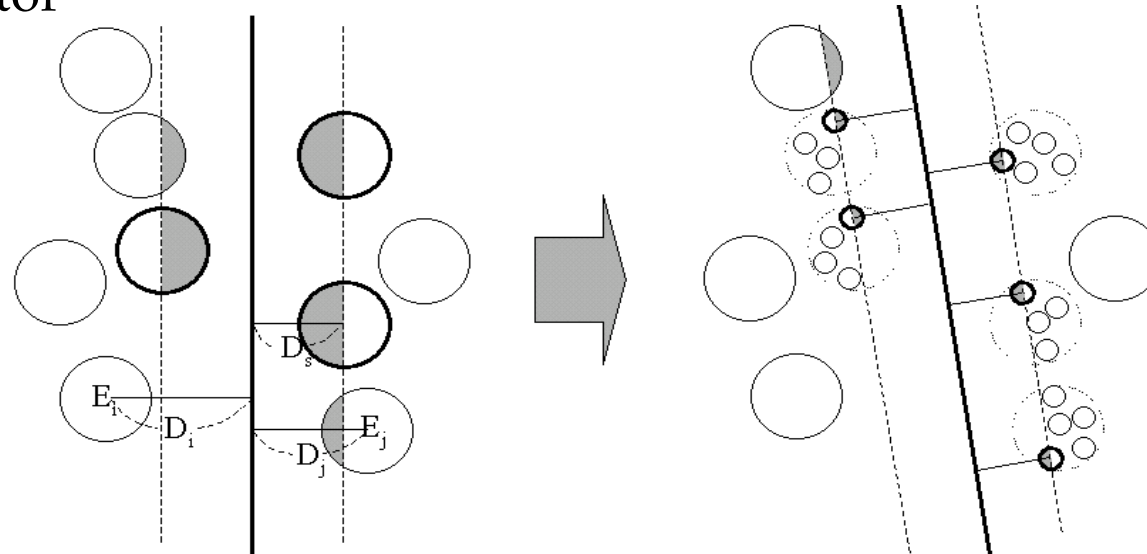
CF-Tree: Hierarchical Micro-cluster



- Read the data set once, construct a statistical summary of the data (i.e., hierarchical clusters) given a limited amount of memory
- Micro-clustering: Hierarchical indexing structure
 - provide finer samples closer to the boundary and coarser samples farther from the boundary

Selective Declustering: Ensure High Accuracy

- CF tree is a suitable base structure for selective declustering
- De-cluster only the cluster E_i such that
 - $D_i - R_i < D_s$, where D_i is the distance from the boundary to the center point of E_i and R_i is the radius of E_i
 - Decluster only the cluster whose subclusters have possibilities to be the support cluster of the boundary
 - “Support cluster”: The cluster whose centroid is a support vector



CB-SVM Algorithm: Outline

- Construct two CF-trees from positive and negative data sets independently
 - Need one scan of the data set
- Train an SVM from the centroids of the root entries
- De-cluster the entries near the boundary into the next level
 - The children entries de-clustered from the parent entries are accumulated into the training set with the non-declustered parent entries
- Train an SVM again from the centroids of the entries in the training set
- Repeat until nothing is accumulated

Accuracy and Scalability on Synthetic Dataset

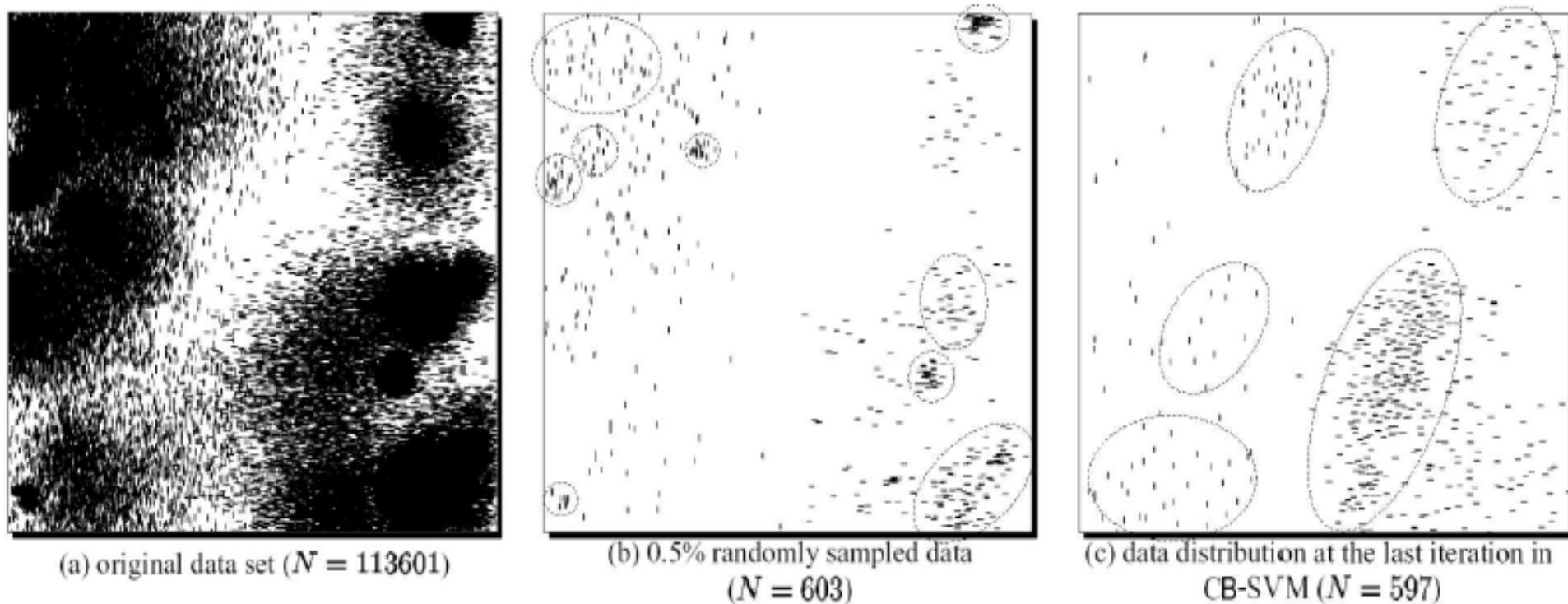


Figure 6: Synthetic data set in a two-dimensional space. '+' : positive data; '-' : negative data

- Experiments on large synthetic data sets shows better accuracy than random sampling approaches and far more scalable than the original SVM algorithm

SVM vs. Neural Network

■ SVM

- Deterministic algorithm
- Nice generalization properties
- Hard to learn – learned in batch mode using quadratic programming techniques
- Using kernels can learn very complex functions

■ Neural Network

- Nondeterministic algorithm
- Generalizes well but doesn't have strong mathematical foundation
- Can easily be learned in incremental fashion
- To learn complex functions—use multilayer perceptron (nontrivial)⁵⁵

SVM Related Links

- SVM Website: <http://www.kernel-machines.org/>
- Representative implementations
 - **LIBSVM**: an efficient implementation of SVM, multi-class classifications, nu-SVM, one-class SVM, including also various interfaces with java, python, etc.
 - **SVM-light**: simpler but performance is not better than LIBSVM, support only binary classification and only in C
 - **SVM-torch**: another recent implementation also written in C

Classification: Basic Concepts

- Classification: Basic Concepts
- Decision Tree Induction
- Bayesian Network
- Support Vector Machine (SVM)
- CF-Tree: Hierarchical Micro-cluster
- Ensemble Learning 
- Random Forest

Ensemble Learning

- Ensemble learning methods : Combining Some ML to improve the results
- It can be simply categorized by the fundamental principle they use to combine multiple models (often called "weak learners") into a single, more powerful model.
 - **1. Averaging Methods**
 - **2. Boosting Methods**
 - **3. Stacking (or Stacked Generalization)**
 - **4. Voting (or Majority Voting)**

1. Averaging Methods

- **Core Idea:** Build several independent models (often of the same type) and average their predictions.
- **How it works:** You train multiple models in parallel, and for a new data point, you take the average (for regression) or the majority vote (for classification) of all their predictions.
- **Key Benefit:** Reduces variance. By averaging, you smooth out the individual errors of each model.
- **Common Examples:**
 - **Bagging** (Bootstrap Aggregating): Training through bootstrap sampling (random sampling with replacement), the classic example is the **Random Forest** algorithm.
 - **Pasting:** Similar to Bagging, but the subsets are drawn without replacement.

Bootstrap Aggregating

- Bootstrap Aggregating, or Bagging, is an ensemble technique designed to improve the stability and accuracy of machine learning models, particularly those like decision trees that have high variance.
- It works by
 - creating **multiple versions of the original training dataset** through bootstrap sampling (**random sampling with replacement**),
 - training a separate base model on each of these new datasets in parallel,
 - and then combining their predictions. For regression tasks, the final prediction is the average of all individual model outputs, while for classification, it is determined by a majority vote.
 - This process reduces overall variance by smoothing out the outputs of individual models, leading to a more robust and reliable predictor that is less likely to overfit to the noise in the training data.

2. Boosting Methods

- **Core Idea:** Build models **sequentially**, where each new model tries to **correct the errors made by the previous ones**.
- **How it works:** Models are trained one after the other.
 - Each new model pays more attention to the training instances that previous models misclassified.
 - Finally, all models are combined through a weighted majority vote.
- **Key Benefit:** Reduces bias and variance, often leading to very high accuracy. The focus is on converting many weak learners into one strong learner.
- **Common Examples:**
 - **AdaBoost** (Adaptive Boosting)
 - **Gradient Boosting** (and its modern implementations like **XGBoost**, **LightGBM**, and **CatBoost**)

3. Stacking (or Stacked Generalization)

- **Core Idea:** Use a machine learning model to learn how to best combine the predictions of several other models.
- **How it works:**
 - Train several different base models (e.g., a Decision Tree, an SVM, a K-NN) on the training data.
 - Have these base models make predictions on a validation set (or via cross-validation).
 - Use these predictions as *new features* to train a final **meta-model** (the blender), which learns which base model to trust and how to combine them best.
- **Key Benefit:** Can capture the unique strengths of different types of models, often leading to superior performance.
- **Common Example:** A stack of a Random Forest, a Logistic Regression, and a Support Vector Machine, with a Linear Model as the final meta-learner.

4. Voting (or Majority Voting)

- **Core Idea:** The simplest form of ensemble. Train multiple, diverse models and let them vote on the final prediction.
- **How it works:** You train several different models (e.g., a Logistic Regression, a Random Forest, and a Neural Network) on the same data. For classification, the final prediction is the one that gets the most votes (**Hard Voting**) or the highest average predicted probability (**Soft Voting**).
- **Key Benefit:** Simple to implement and understand. Often performs better than any single model in the ensemble.

Classification: Basic Concepts

- Classification: Basic Concepts
- Decision Tree Induction
- Bayesian Network
- Support Vector Machine (SVM)
- CF-Tree: Hierarchical Micro-cluster
- Ensemble Learning
- Random Forest 

Random Forest

- **The "Wisdom of the Crowd" for Trees**
- Imagine you want to predict what movie you'll like, but you're not sure. Instead of asking one friend (a single expert), you ask a whole crowd of friends (an ensemble).
- A **Random Forest** does exactly this, but with Decision Trees.

How it works

- 1. Build Many "Mini-Experts":** It creates a whole bunch of Decision Trees. Each tree is trained on a slightly different, random piece of the training data (this is called **bagging** or **bootstrap aggregating**).
- 2. Add Extra Randomness:** When each tree is making a decision, it's not allowed to look at all the features. It must pick from a random subset. This forces each tree to be unique and learn different patterns, preventing them from all making the same mistake.
- 3. Take a Vote:** When it's time to make a final prediction (e.g., "Is this email spam?"), the Random Forest asks *every single tree* for its answer. The final result is whatever answer gets the most votes.

Simple Analogy

- Think of a Random Forest as a game show's "Ask the Audience" lifeline.
- A single tree is like asking one smart person in the audience.
- A Random Forest is asking **the entire audience** and going with the most popular answer.
- Even if some people in the audience are wrong, the collective opinion of the whole crowd is very often correct. The Random Forest uses this "wisdom of the crowd" to create a model that is much more accurate and stable than any single tree could be.

Question & Answer
